

PROJECT REPORT

PROJECT TITLE : Delhi Smart City Transportation analysis

1. INTRODUCTION :

This project, titled "**Delhi Smart City Transportation analysis**," leverages the power of Amazon Web Services (AWS) to build a robust, serverless data lakehouse. The objective is to provide a unified analytical framework that uncovers hidden correlations between the city's environment and its mobility. By centralizing disparate data silos into a single source of truth, this system enables the identification of critical safety hotspots and congestion patterns, ultimately aiming to foster a safer, more efficient, and sustainable urban environment for the citizens of Delhi.

2. PROBLEM STATEMENT :

Urban transportation systems in Delhi face significant challenges including traffic congestion, road accidents, and public transport delays. These issues are influenced by dynamic factors such as traffic volume, infrastructure constraints, and weather conditions. Traditional traffic analysis methods often rely on isolated datasets and manual reporting, which fail to provide integrated insights. There is a need for a scalable data engineering framework that can process and integrate multiple structured CSV datasets (traffic, accidents, pollution and weather) to generate actionable insights for smart city planning and traffic management.

3. LITERATURE REVIEW :

Link	Author(s)	Year	Method Used	Key Contribution
https://www.sciencedirect.com/science/article/abs/pii/S0045790624005512	Jing Zhao	2024	Urban Transportation Data Analysis & Intelligent Transport Systems	Comprehensive review highlighting multi-source transportation data integration for smart city planning.
https://www.mdpi.com/2624-6511/7/1/10	Tsalikidis et al.	2024	Multi-step Traffic Forecasting with Weather Integration	Demonstrated that combining traffic sensor data with weather data improves congestion prediction.
https://ijai.iaescor.e.com/index.php/IJAI/article/view/21322	Al-Selwi et al.	2022	Weather-Enriched Traffic Flow Modeling	Validated that temperature, rainfall, and wind speed significantly influence traffic flow patterns.

https://link.springer.com/article/10.1007/s41019-020-00151-z	Haitao Yuan	2021	GPS Data Mining for Urban Mobility	Identified urban mobility patterns using large-scale GPS trajectory data.
---	-------------	------	------------------------------------	---

4. OBJECTIVES :

Establishing an automated ETL pipeline to convert raw transit telemetry into actionable insights.

- Scalable serverless architecture
- Automated data cleansing
- Real-time executive dashboards

5. DATASET DESCRIPTION :

The project utilizes a multi-domain synthetic urban dataset designed to simulate real-world smart city analytics in Delhi. The dataset spans a defined temporal range covering first half (Jan-Jun 2024), and is composed of four distinct datasets representing different aspects of urban transportation and environmental conditions.

A. Weather Dataset (**weather.csv**)

This dataset contains meteorological attributes that influence traffic flow, pollution levels, and accident conditions:

- **date**: Observation date (standardized format YYYY-MM-DD).
- **temp_c**: Temperature in degrees Celsius.
- **humidity**: Atmospheric humidity percentage.
- **windspeed_kph**: Wind speed in kilometers per hour.
- **precipitation**: Precipitation level in millimeters.
- **heat_index**: Heat index of the day.
- **condition_text**: Weather condition (Clear, Partially cloudy, Rain, etc.).

Purpose:

To analyze how weather conditions impact traffic congestion, accident severity, and pollution levels.

B. Traffic Dataset (**traffic.csv**)

This dataset represents road traffic patterns and congestion behavior across different time periods:

- **date**: Date of traffic observation.
- **location**: Area name aligned with other datasets for integration.
- **time**: Time of observation (Morning, Evening, Night, Afternoon).
- **traffic_density_index**: Traffic density score.
- **vehicle_count**: No. of vehicles on road per day.
- **road_type**: Type of road (Highway, Main Road, Arterial).

- **average_speed_kmph**: Average vehicle speed.
- **congestion_level**: Congestion intensity (low, moderate, heavy, severe, etc)

Purpose:

To study traffic patterns and their relationship with congestion, speed, and pollution.

C. Accident Dataset (**accidents.csv**)

This dataset captures road accident details and severity indicators:

- **accident_id**: unique id for each accident for recognition.
- **date**: Date of accident occurrence.
- **time**: timing of the day(morning, evening, afternoon, night).
- **location**: Area where accident occurred.
- **vehicle_at_fault**: vehicle that has been damaged.
- **victim**: person who is injured.
- **type_of_accident**: how the accident occurred.
- **injured**: Number of injured individuals.
- **killed**: Number of fatalities.
- **accident_severity**: Severity classification (Minor, Moderate, Severe).

Purpose:

To analyze accident trends and evaluate the influence of traffic and environmental factors on road safety.

D. Pollution Dataset (**pollution.csv**)

This dataset contains air quality indicators critical for environmental analysis:

- **date**: Date of pollution measurement.
- **pm2_5**: Fine particulate matter concentration.
- **pm10**: Coarse particulate matter concentration.
- **no2**: Nitrogen dioxide levels.
- **so2**: sulphur dioxide levels.
- **co**: Carbon monoxide levels.
- **aqi**: Air Quality Index derived from pollutants.

Purpose:

To evaluate air quality patterns and correlate pollution with traffic and weather conditions.

6. SYSTEM ARCHITECTURE :

The architecture follows a decoupled serverless model :

1. **Storage Layer** : Amazon S3 (Simple Storage Service)
2. **Processing Layer** : AWS Lambda (Python, Pandas, AWS Wrangler)
3. **Analytical Layer** : Amazon Athena (Serverless SQL)
4. **Visualization Layer** : Amazon QuickSight (Business Intelligence)

7. PHASES OF IMPLEMENTATION :

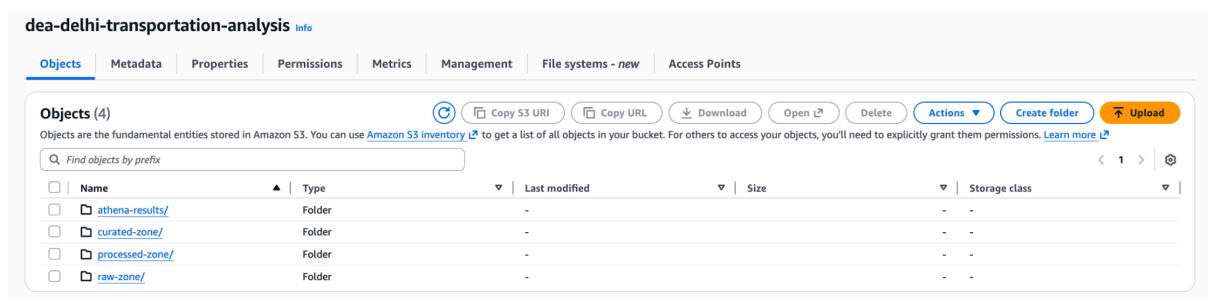
Phase 1 : Data storage Layer (Amazon S3)

The foundation of the Delhi Smart City Transportation project is a scalable, distributed Data Lake built on **Amazon S3**. We implemented a tiered storage strategy to manage the lifecycle of the transportation data, ensuring high durability and organized data flow.

a) S3 Bucket Structure: The Medallion Pattern

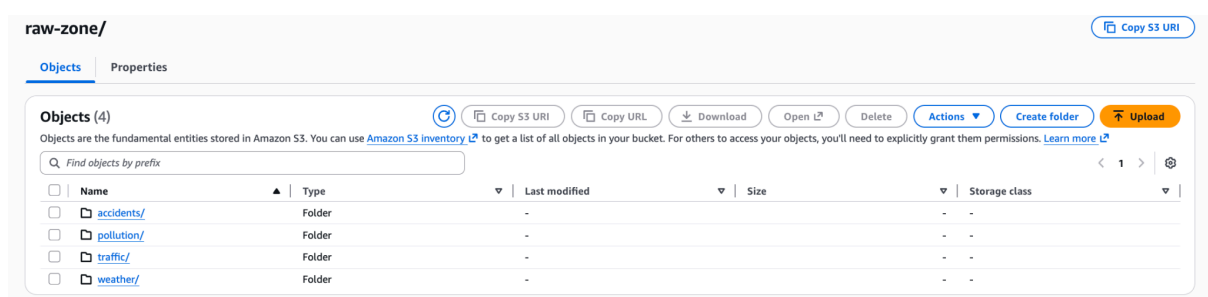
As seen in the **dea-delhi-transportation-analysis bucket**, we have organized the storage into four distinct functional zones:

1. **raw-zone/**: The landing zone for immutable, original datasets. No transformations are performed here; it serves as the "Single Source of Truth."
2. **processed-zone/**: Stores data after initial cleaning, type casting, and geo-fencing (performed by AWS Lambda).
3. **curated-zone/**: Contains high-value, business-ready data, typically stored in columnar formats (like Parquet) to optimize for analytical performance.
4. **athena-results/**: A dedicated location to store the output of SQL queries and metadata generated during the analysis phase.



b) Ingestion and Dataset taxonomy

Within the **raw-zone/**, the project handles four critical streams of transportation-related data. Each stream is categorized into its own folder to facilitate automated partitioning and ease of access:



- **Accidents (accidents.csv)**: Records regarding incident locations, severity, and timestamps to identify high-risk traffic zones.
- **Pollution (pollution.csv)**: Air quality index (AQI) logs captured from transit-adjacent sensors to correlate traffic density with environmental impact.

- **Traffic (traffic.csv)**: The primary telemetry stream containing vehicle IDs, GPS coordinates, instantaneous speeds, and vehicle types.
- **Weather (weather.csv)**: Historical weather conditions (rain, visibility, temperature) used to analyze how environmental factors influence Delhi's congestion.

The following table summarizes the data files shown in the screenshots:

Folder	File Name	Type	Last Modified	Size	Storage Class
accidents/	accidents.csv	csv	April 17, 2026, 18:42:21 (UTC+05:30)	102.2 KB	Standard
pollution/	pollution.csv	csv	April 17, 2026, 18:42:58 (UTC+05:30)	8.1 KB	Standard
traffic/	traffic.csv	csv	April 17, 2026, 18:43:12 (UTC+05:30)	43.1 KB	Standard
weather/	weather.csv	csv	April 17, 2026, 18:43:28 (UTC+05:30)	10.2 KB	Standard

Phase 2 : Data processing layer (AWS Lambda)

The Processing Layer acts as the "engine" of the pipeline. It is responsible for transforming raw, noisy IoT telemetry into clean, structured datasets ready for analysis. This project utilizes a Serverless ETL approach, ensuring that the system only consumes resources when new data arrives.

a) Serverless execution logic :

We deployed a Python-based AWS Lambda function (**delhi_etl_lambda**) that is configured with an S3 Event Trigger.

- Mechanism: As soon as a **.csv** file is uploaded to the **raw-zone/** prefix, S3 sends a JSON event to Lambda.
- Automation: This eliminates the need for manual intervention or scheduled cron jobs, making the pipeline fully reactive.

delhi_etl_lambda

▼ Function overview [Info](#)

Diagram | Template



delhi_etl_lambda



Layers



S3

+ Add trigger

+ Add destination

b) Dataset-specific transformation logic :

The Lambda function is context-aware and applies cleaning only to relevant numerical columns based on the file type:

- Accident Data: Targets **accident_severity** and **vehicle_at_fault**.
- Traffic Data: Targets **average_speed_kmph**, **Location** and **vehicle_count**.
- Weather Data: Targets **temp_c**, **humidity**, and **condition**.
- Pollution Data: Targets **aqi**, **pm2_5**, and **pm10**.

```
1 import aws wrangler as wr
2 import pandas as pd
3 import numpy as np
4 import logging
5
6 # Set up logging for CloudWatch
7 logger = logging.getLogger()
8 logger.setLevel(logging.INFO)
9
10 def lambda_handler(event, context):
11     try:
12         # 1. Identify Bucket and File Key
13         bucket = event['Records'][0]['s3']['bucket']['name']
14         key = event['Records'][0]['s3']['object']['key']
15
16         # 2. Determine paths (Raw to Processed)
17         input_path = f"s3://{bucket}/{key}"
18         output_path = f"s3://{bucket}/{key.replace('raw-zone', 'processed-zone')}"
19
20         # Identify the folder name to determine the dataset type
21         # Expected structure: raw-zone/folder_name/file.csv
22         path_parts = key.split('/')
23         dataset_type = path_parts[1].lower()
24
25         logger.info(f"Cleaning {dataset_type} dataset from {input_path}")
26
27         # 3. Read CSV using aws wrangler
28         df = wr.s3.read_csv(path=input_path)
```

CodeTestMonitorConfigurationAliasesVersions

Code sourceInfo

Open in Visual Studio CodeUpload

← →delhi_etl_lambda

EXPLORER

DELHI_ETL_LAMBDA

lambda_function.py

DEPLOYCurrent

Deploy (⌘U)Test (⌘I)

TEST EVENTS [SELECTED: TESTS3UPLOAD]

Create new test eventPrivate saved eventsTestS3Upload

ENVIRONMENT VARIABLES

lambda_function.py

10def lambda_handler(event, context):

11try:

30# 4. Dataset-Specific Cleaning Logic

31if "traffic" in dataset_type:

32df.loc[(df['Average_Speed_kmph'] > 150) | (df['Average_Speed_kmph'] < 0), 'Average_Speed_kmph'] = df['Average_Speed_kmph'].median()

33df['Road_Category'] = df['Road_Category'].replace({'main road': 'Main Road', 'High-Way': 'Highway', 'Artrial': 'Arterial'})

34df = df.dropna(subset=['Location', 'Vehicle_Count'])

35

36

37

38elif "accident" in dataset_type:

39df.loc[df['Killed'] > 0, 'Accident_severity'] = 'fatal'

40df['Injured'] = df['Injured'].abs() # Ensure no negative injuries

41df['Vehicle_at_fault'] = df['Vehicle_at_fault'].replace({'car': 'Private Car', '2-wheeler': 'Two-Wheeler', 'BUSS': 'Bus'})

42

43

44elif "weather" in dataset_type:

45# 1. Cap Humidity

46df.loc[df['humidity'] > 100, 'humidity'] = 100

47

48# 2. Fix Precip for Clear days (Ensure brackets match exactly)

49df.loc[(df['conditions'].str.contains('Clear', case=False)) & (df['precip'] > 10), 'precip'] = 0

50

51# 3. Filter extreme temp

52df.loc[(df['temp'] > 60) | (df['temp'] < -10), 'temp'] = df['temp'].median()

53

54# 4. Keep only first condition

55df['conditions'] = df['conditions'].str.split(',').str[0].str.strip()

56

57elif "pollution" in dataset_type:

58mask = df['PM2.5'] > df['PM10'] # Rule: PM10 must be >= PM2.5

59df.loc[mask, 'PM10'] = df.loc[mask, 'PM2.5'] + 5

60df.loc[df['CO'] < 0, 'CO'] = df['CO'].median()

61df.loc[(df['AQI'] <= 0) & (df['PM10'] > 100), 'AQI'] = df['AQI'].replace(0, df['AQI'].median())

CodeTestMonitorConfigurationAliasesVersions

Code sourceInfo

Open in Visual Studio CodeUpload

← →delhi_etl_lambda

EXPLORER

DELHI_ETL_LAMBDA

lambda_function.py

DEPLOYCurrent

Deploy (⌘U)Test (⌘I)

TEST EVENTS [SELECTED: TESTS3UPLOAD]

Create new test eventPrivate saved eventsTestS3Upload

ENVIRONMENT VARIABLES

lambda_function.py

10def lambda_handler(event, context):

11try:

60

61# 5. Write Cleaned Data to Processed Zone

62wr.s3.to_csv(df=df, path=output_path, index=False)

63

64logger.info(f"Successfully moved cleaned data to {output_path}")

65return {"status": "success", "processed_file": output_path}

66

67except Exception as e:

68logger.error(f"Error: {str(e)}")

69raise e

ConfigurationAliasesVersions

Triggers (1)Info

Fix errorsEditDeleteAdd trigger

Search for triggers

< 1 >

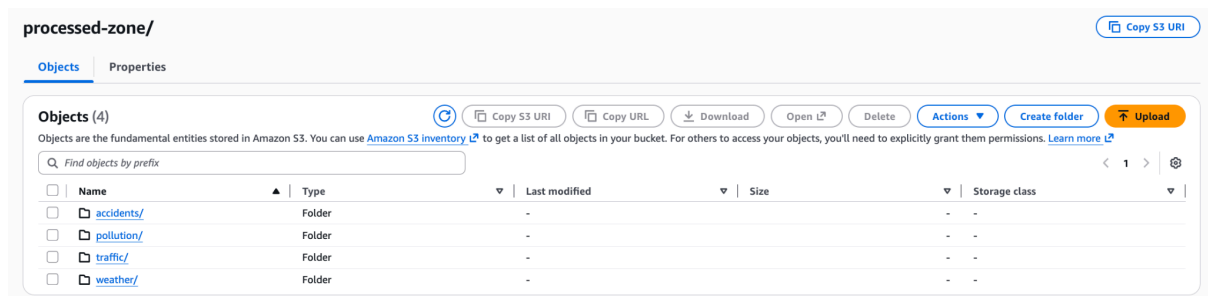
Trigger

S3: dea-delhi-transportation-analysis

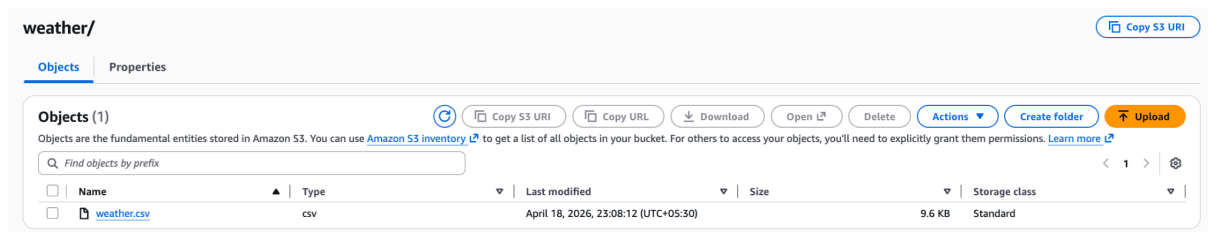
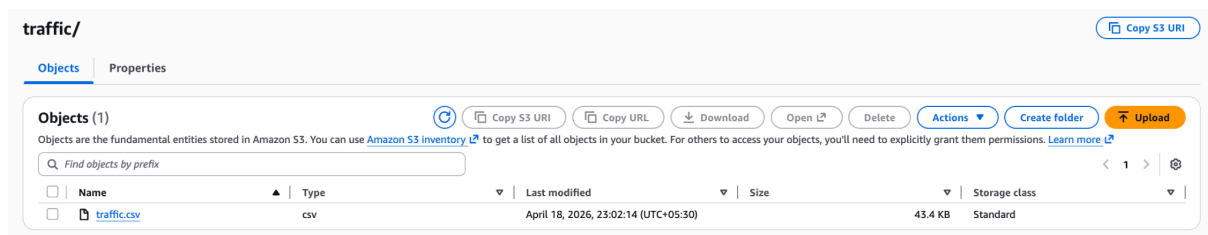
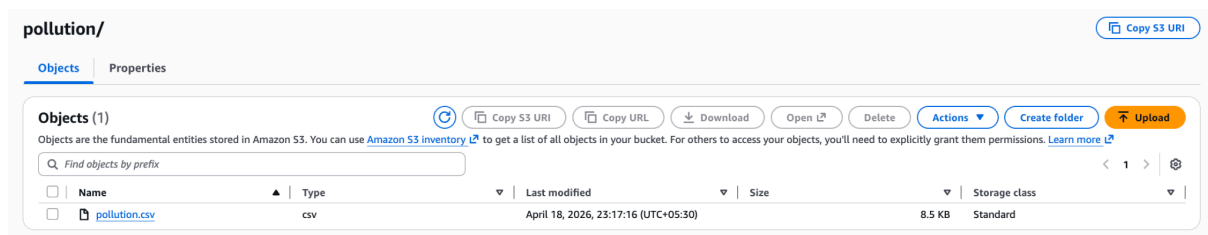
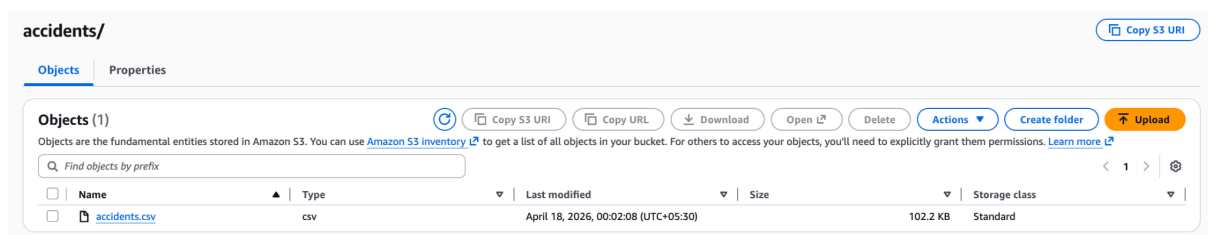
arn:aws:s3:::dea-delhi-transportation-analysis

Details

c) **Path Re-routing:** Dynamically change the S3 key from the **raw-zone/** to the **processed-zone/**.



d) **Integrity Check:** Print a final log showing the number of rows removed and the successful path of the finalized clean data.



Phase 3 : Data Analytical Layer (Amazon Athena)

The Analytical Layer transitions the project from raw data engineering to urban intelligence. After the data was cleaned and staged in the **processed-zone/**, we utilized Amazon Athena—a serverless interactive query service—to perform complex SQL-based analysis

directly on the S3 data lake.

a) Data cataloging and schema definition

We first established a structured environment by creating a dedicated database, **delhi_transport_analysis_db**.

- External Tables: We defined external tables for **traffic**, **weather**, **pollution**, and **accidents**.
- Schema-on-Read: By defining the schema (column names and data types like **INT**, **DATE** and **DOUBLE**) in Athena, we were able to run standard SQL queries on our CSV files without needing to load them into a traditional database.

The screenshot shows the AWS Athena console interface. On the left, the 'Data' sidebar is visible with the 'Database' set to 'delhi_transport_analysis_db'. The 'Tables and views' section lists five tables: delhi_accidents, delhi_pollution, delhi_traffic, delhi_weather, and new_master_dataset. The main editor area displays the SQL code for creating the 'delhi_accidents' external table. The code is as follows:

```
1 CREATE EXTERNAL TABLE IF NOT EXISTS delhi_accidents (  
2   accident_id STRING,  
3   date DATE,  
4   time STRING,  
5   location STRING,  
6   vehicle_type_at_fault STRING,  
7   victim STRING,  
8   type_of_accident STRING,  
9   killed INT,  
10  injured INT,  
11  accident_severity STRING  
12 )  
13 ROW FORMAT DELIMITED  
14 FIELDS TERMINATED BY ','  
15 STORED AS TEXTFILE  
16 LOCATION 's3://dea-delhi-transportation-analysis/processed-zone/accidents/'  
17 TBLPROPERTIES ('skip.header.line.count'='1');
```

Below the SQL code, the 'Query results' section shows a green bar indicating the query is 'Completed' and a message stating 'Query successful.'.

The screenshot shows the AWS Athena console interface. On the left, the 'Data' sidebar is visible with the 'Database' set to 'delhi_transport_analysis_db'. The 'Tables and views' section lists five tables: delhi_accidents, delhi_pollution, delhi_traffic, delhi_weather, and new_master_dataset. The main editor area displays the SQL code for creating the 'delhi_traffic' external table. The code is as follows:

```
1 CREATE EXTERNAL TABLE IF NOT EXISTS delhi_transport_analysis_db.delhi_traffic (  
2   date date,  
3   time string,  
4   location string,  
5   vehicle_count int,  
6   average_speed_kmph int,  
7   traffic_density_index double,  
8   congestion_level string,  
9   road_category string  
10 )  
11 ROW FORMAT DELIMITED FIELDS TERMINATED BY ','  
12 STORED AS TEXTFILE  
13 LOCATION 's3://dea-delhi-transportation-analysis/processed-zone/traffic/'  
14 TBLPROPERTIES ('skip.header.line.count'='1');
```

Below the SQL code, the 'Query results' section shows a green bar indicating the query is 'Completed' and a message stating 'Query successful.'.

EditorRecent queriesSaved queriesQuery settings

Data

↻

<

Data source

AwsDataCatalog

Catalog

None

Database

delhi_transport_analysis_db

Tables and views

Create

⊞

Filter tables and views

Tables (5)

<

1

>

delhi_accidents

delhi_pollution

delhi_traffic

delhi_weather

new_master_dataset

Views (0)

<

1

>

Query 1 : XQuery 2 : XQuery 4 : XQuery 5 : XQuery 6 : XQuery 7 : X

1

CREATE EXTERNAL TABLE IF NOT EXISTS delhi_transport_analysis_db.delhi_pollution (

2

date date,

3

pm2_5 double,

4

pm10 double,

5

no2 double,

6

so2 double,

7

co double,

8

aqi int

9

)

10

ROW FORMAT DELIMITED FIELDS TERMINATED BY ','

11

STORED AS TEXTFILE

12

LOCATION 's3://dea-delhi-transportation-analysis/processed-zone/pollution/'

13

TBLPROPERTIES ('skip.header.line.count'='1');

SQL

Ln 1, Col 1

Run again

Explain

Cancel

Clear

Create

Query results

Query stats

Completed

Query successful.

EditorRecent queriesSaved queriesQuery settings

Data

↻

<

Data source

AwsDataCatalog

Catalog

None

Database

delhi_transport_analysis_db

Tables and views

Create

⊞

Filter tables and views

Tables (5)

<

1

>

delhi_accidents

delhi_pollution

delhi_traffic

delhi_weather

new_master_dataset

Views (0)

<

1

>

Query 1 : XQuery 2 : XQuery 4 : XQuery 5 : XQuery 6 : XQuery 7 : X

1

CREATE EXTERNAL TABLE IF NOT EXISTS delhi_transport_analysis_db.delhi_weather (

2

date date,

3

temp_c double,

4

humidity double,

5

precipitation double,

6

windspeed_kph double,

7

heat_index double,

8

condition_text string

9

)

10

ROW FORMAT DELIMITED FIELDS TERMINATED BY ','

11

STORED AS TEXTFILE

12

LOCATION 's3://dea-delhi-transportation-analysis/processed-zone/weather/'

13

TBLPROPERTIES ('skip.header.line.count'='1');

SQL

Ln 1, Col 1

Run again

Explain

Cancel

Clear

Create

Query results

Query stats

Completed

Query successful.

b) Processed data retrieved

Query 1 : XQuery 2 : XQuery 4 : XQuery 5 : XQuery 6 : XQuery 7 : XQuery 9 : XQuery 10 : XQuery 11 : XQuery 12 : X

1

SELECT * FROM "delhi_transport_analysis_db"."delhi_accidents" limit 10;

SQL

Ln 1, Col 61

Run again

Explain

Cancel

Clear

Create

Reuse query results up to 60 minutes ago

Query results

Query stats

Completed

Time in queue: 102 msRun time: 536 msData scanned: 102.22 KB

Results (10)

Copy

Download results CSV

Search rows

#	accident_id	date	time	location	vehicle_type_at_fault	victim	type_of_accident	killed	injured	accident_severity
1	DEL-ACC-00101	2024-01-01	morning	Pitampura	E-Rickshaw	Passenger	Overturning	0	0	minor
2	DEL-ACC-00102	2024-01-01	afternoon	Rajouri Garden	Two-Wheeler	Passenger	Head-on Collision	0	1	moderate
3	DEL-ACC-00103	2024-01-01	evening	Okhla	Private Car	Cyclist	Skidding	0	0	minor
4	DEL-ACC-00104	2024-01-01	evening	Okhla	Bus	Pedestrian	Side-swipe	0	1	moderate
5	DEL-ACC-00105	2024-01-01	night	Chandni Chowk	Auto Rickshaw	Driver	Side-swipe	0	0	minor
6	DEL-ACC-00106	2024-01-02	morning	Janakpuri	Truck	Pedestrian	Hit and Run	1	2	fatal
7	DEL-ACC-00107	2024-01-02	morning	Janakpuri	E-Rickshaw	Driver	Overturning	0	1	moderate
8	DEL-ACC-00108	2024-01-02	afternoon	Pitampura	Auto Rickshaw	Pedestrian	Hit and Run	0	2	serious
9	DEL-ACC-00109	2024-01-02	evening	Patparganj	E-Rickshaw	Cyclist	Head-on Collision	0	2	serious
10	DEL-ACC-00110	2024-01-02	evening	Patparganj	E-Rickshaw	Driver	Overturning	0	2	serious

Query 1Query 2Query 4Query 5Query 6Query 7Query 9Query 10Query 11Query 12

1 SELECT * FROM "delhi_transport_analysis_db"."delhi_traffic" limit 10;

SQLLn 1, Col 59

Run again

Explain

Cancel

Clear

Create

Reuse query results
up to 60 minutes ago

Query results

Query stats

Completed

Time in queue: 116 msRun time: 453 msData scanned: 43.36 KB

Results (10)

Copy

Download results CSV

Search rows

#	date	time	location	vehicle_count	average_speed_kmph	traffic_density_index	congestion_level	road_category
1	2024-01-01	morning	Pitampura	1075	27	38.39	Heavy	Arterial
2	2024-01-01	afternoon	Rajouri Garden	648	41	15.43	Moderate	Main Road
3	2024-01-01	evening	Okhla	1582	23	65.92	Severe	Highway
4	2024-01-01	night	Chandni Chowk	187	66	2.79	Low	Highway
5	2024-01-02	morning	Janakpuri	1173	25	45.12	Severe	Main Road
6	2024-01-02	afternoon	Pitampura	652	39	16.3	Moderate	Main Road
7	2024-01-02	evening	Patparganj	1496	25	57.54	Severe	Main Road
8	2024-01-02	night	Chandni Chowk	317	70	4.46	Low	Main Road
9	2024-01-03	morning	Chandni Chowk	934	27	33.36	Heavy	Arterial
10	2024-01-03	afternoon	Dwarka	751	41	17.88	Moderate	Highway

Query 1Query 2Query 4Query 5Query 6Query 7Query 9Query 10Query 11Query 12

1 SELECT * FROM "delhi_transport_analysis_db"."delhi_pollution" limit 10;

SQLLn 1, Col 61

Run again

Explain

Cancel

Clear

Create

Reuse query results
up to 60 minutes ago

Query results

Query stats

Completed

Time in queue: 125 msRun time: 397 msData scanned: 8.52 KB

Results (10)

Copy

Download results CSV

Search rows

#	date	pm2_5	pm10	no2	so2	co	aqi
1	2024-01-01	71.18	288.33	24.09	21.92	0.74	244
2	2024-01-02	76.61	290.95	26.95	22.19	0.64	259
3	2024-01-03	81.94	308.65	26.41	22.01	1.03	246
4	2024-01-04	86.58	349.36	21.61	22.18	1.02	324
5	2024-01-05	64.61	267.06	19.8	22.36	0.85	246
6	2024-01-06	78.21	319.32	20.7	22.47	0.8	220
7	2024-01-07	80.76	334.34	27.08	22.9	0.77	308
8	2024-01-08	95.24	373.77	24.92	22.87	0.85	314
9	2024-01-09	52.82	212.74	20.55	22.8	0.69	254
10	2024-01-10	73.62	286.79	22.23	22.86	0.7	168

Query 1Query 2Query 4Query 5Query 6Query 7Query 9Query 10Query 11Query 12

1 SELECT * FROM "delhi_transport_analysis_db"."delhi_weather" limit 10;

SQLLn 1, Col 59

Run again

Explain

Cancel

Clear

Create

Reuse query results
up to 60 minutes ago

Query results

Query stats

Completed

Time in queue: 107 msRun time: 411 msData scanned: 9.64 KB

Results (10)

Copy

Download results CSV

Search rows

#	date	temp_c	humidity	precipitation	windspeed_kph	heat_index	condition_text
1	2024-01-01	12.1	81.7	0.0	11.2	303.6701381	Clear
2	2024-01-02	11.2	82.2	0.0	11.2	311.7059851	Clear
3	2024-01-03	10.8	87.8	0.0	11.2	315.6345443	Clear
4	2024-01-04	10.1	92.7	0.0	13.0	320.306828	Partially cloudy
5	2024-01-05	12.0	92.9	0.0	11.2	302.174724	Partially cloudy
6	2024-01-06	12.1	84.9	0.0	11.2	304.1238211	Clear
7	2024-01-07	11.6	81.2	0.0	7.6	307.8596363	Partially cloudy
8	2024-01-08	10.9	88.1	0.0	7.6	314.646701	Clear
9	2024-01-09	10.9	90.8	0.0	13.0	313.7108866	Partially cloudy
10	2024-01-10	9.7	87.4	0.0	14.8	325.9666362	Clear

c) Data integration : The master query table

The highlight of this phase was the creation of a Master Query Table using a CTAS (Create Table As Select) statement. In urban planning, data is most valuable when it is correlated.

The Master Table Strategy: We performed a complex JOIN operation across our four primary datasets. By joining the tables on common dimensions—specifically Timestamp and Geographic Zone—we created a unified view of the city's status.

- **Unified Metrics:** A single row in the Master Table now shows the traffic speed, PM2.5 levels, temperature, and accident status for a specific area at a specific time.
- **Analytical Efficiency:** This "denormalized" approach is a best practice in data warehousing. It ensures that our final visualization tool (QuickSight) doesn't have to perform expensive joins in real-time, leading to much faster dashboard performance.

The screenshot displays a data editor interface with a sidebar on the left and a main query editor on the right. The sidebar includes sections for 'Data' (Data source: AwsDataCatalog, Catalog: None, Database: delhi_transport_analysis_db), 'Tables and views' (listing delhi_accidents, delhi_pollution, delhi_traffic, delhi_weather, and new_master_dataset), and 'Views (0)'. The main editor shows a SQL query for creating a table named 'delhi_transport_analysis_db.new_master_dataset'. The query uses a CTAS (Create Table As Select) statement, selecting data from four tables: delhi_traffic, delhi_accidents, delhi_weather, and delhi_pollution, joined on common dimensions (Date, Time, Location). The query is as follows:

```
1 CREATE TABLE delhi_transport_analysis_db.new_master_dataset
2 - WITH (
3   format = 'PARQUET',
4   external_location = 's3://dea-delhi-transportation-analysis/curated-zone/master_table/'
5 ) AS
6 SELECT
7   -- Traffic Data (Base)
8   t.Date,
9   t.Time,
10  t.Location,
11  t.Vehicle_Count,
12  t.Average_Speed_kmph,
13  t.Traffic_Density_Index,
14  t.Congestion_Level,
15  t.Road_Category,
16
17  -- Accident Data
18  a.Accident_id,
19  a.vehicle_type_at_fault,
20  a.Victim,
21  a.Type_of_accident,
22  COALESCE(a.Killed, 0) as Killed,
23  COALESCE(a.Injured, 0) as Injured,
24  a.Accident_severity,
25
26  -- Weather Data (Daily)
27  w.temp_c,
28  w.humidity,
29  w.precipitation,
30  w.windspeed_kph,
31  w.condition_text as Weather_Condition,
32
33  -- AQI Data (Daily)
34  p.PM2_5,
35  p.PM10,
36  p.AQI
37
38 FROM delhi_transport_analysis_db.delhi_traffic t
39 LEFT JOIN delhi_transport_analysis_db.delhi_accidents a
40   ON t.Date = a.Date
41   AND t.Time = a.Time
42   AND t.Location = a.Location
43 LEFT JOIN delhi_transport_analysis_db.delhi_weather w
44   ON t.Date = w.Date
45 LEFT JOIN delhi_transport_analysis_db.delhi_pollution p
46   ON t.Date = p.DATE;
```

Below the query editor, there are buttons for 'Run again', 'Explain', 'Cancel', 'Clear', and 'Create'. The 'Query results' tab is active, showing a green bar indicating 'Completed' and the message 'Query successful.'

- **curated-zone:** master_dataset_final is created inside the curated-zone folder as “The gold layer” of the dataset. The cleanest form of the dataset.

curated-zone/ Copy S3 URI

Objects Properties

Objects (1) Copy S3 URI Copy URL Download Open L Delete Actions Create folder Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Name	Type	Last modified	Size	Storage class
master_table/	Folder	-	-	-

master_table/ Copy S3 URI

Objects Properties

Objects (1) Copy S3 URI Copy URL Download Open L Delete Actions Create folder Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Name	Type	Last modified	Size	Storage class
20260418_181950_00133_2vjgq_be717e4e-12fd-4e34-95eb-f0b668e9f9be	-	April 18, 2026, 23:49:52 (UTC+05:30)	24.5 KB	Standard

d) Results & Query analysis

With the Master Table ready, we executed several deep-dive queries to extract insights.

Data Query 2 Query 4 Query 5 Query 6 Query 7 Query 9 Query 10 Query 11 Query 12

Data source: AwsDataCatalog
Catalog: None
Database: delhi_transport_analysis_db

Tables and views Create

Filter tables and views

Tables (5): delhi_accidents, delhi_pollution, delhi_traffic, delhi_weather, new_master_dataset

Views (0)

Query 12:

```
1 SELECT
2   Location,
3   AVG(PM2_5) AS Avg_PM2_5,
4   AVG(PM10) AS Avg_PM10,
5   AVG(AQI) AS Avg_AQI,
6   MAX(AQI) AS Peak_Pollution_Recorded,
7   COUNT(DISTINCT Date) AS Days_of_Data
8 FROM delhi_transport_analysis_db.new_master_dataset
9 GROUP BY Location
10 ORDER BY Avg_AQI DESC;
```

SQL Ln 1, Col 1

Run again Explain L Cancel Clear Create Reuse query results up to 60 minutes ago

Query results Query stats

Completed Time in queue: 104 ms Run time: 561 ms Data scanned: 4.88 KB

Results (20) Copy Download results CSV

Search rows

#	Location	Avg_PM2_5	Avg_PM10	Avg_AQI	Peak_Pollution_Recorded	Days_of_Data
1	Greater Kailash	166.4783690994202	335.3881418266929	248.77272727272728	383	25
2	Pitampura	102.03329623032778	281.1700819446135	240.01785714285714	383	34
3	Okhla	165.34548445742337	323.8783146461026	236.47169811320754	383	31
4	Karol Bagh	123.34938220164408	283.5414276561895	234.11363636363637	378	29
5	Vikas Puri	101.83131221650945	269.71508033245146	233.43478260869566	353	37
6	Janakpuri	90.68417143509947	261.59062304800267	227.2258064516129	383	33
7	Paschim Vihar	128.7108167402547	277.0120667402547	224.546875	367	32
8	Saket	117.97227251957825	288.4932894687308	224.3050847457627	367	30

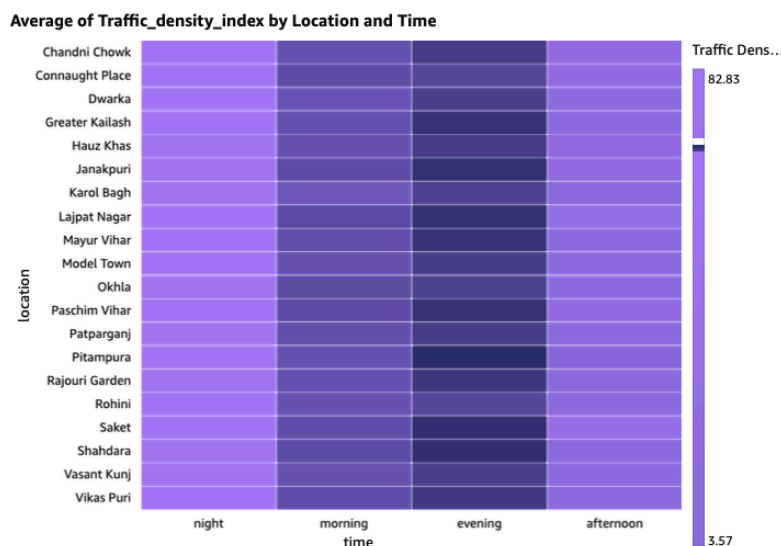
Phase 4 : Data Visualisation Layer

a) Spatiotemporal Analysis of Traffic Density across Delhi

- **Dimensional Correlation:** The chart plots Location (Y-axis) against Time of Day (X-axis), using color saturation to represent the **Average Traffic_density_index**.
- **Intensity Scale:** Darker shades indicate peak congestion levels (reaching up to 82.83), while lighter purple hues represent fluid traffic movement (as low as 3.57).

Insights

- **Peak Period Identification:** A clear "Congestion Corridor" is visible during the Morning and Evening shifts across almost all locations, confirming traditional rush-hour patterns likely linked to office commutes.
- **Location-Specific Variance:** Areas like Janakpuri, Pitampura, and Saket show significantly deeper saturation in the evening compared to the morning, suggesting these are major commercial or transit hubs where outgoing traffic bottlenecks late in the day.



b) Distribution of Traffic Accidents by Severity Level

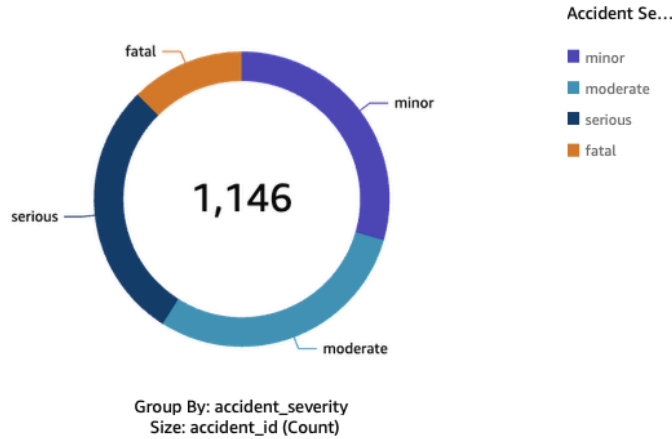
- **Total Volume:** The central metric shows a total of 1,146 recorded accident incidents across the dataset.
- **Proportional Breakdown:** The chart segments these incidents into four categories: minor, moderate, serious, and fatal, based on the impact of the collision.

Insights

- **Dominance of High-Risk Incidents:** A significant portion of the chart is occupied by serious and moderate accidents, suggesting that a majority of collisions in these zones result in more than just minor vehicle damage.

- **Critical Safety Gap:** The presence of a notable fatal accident slice indicates an urgent need for enhanced speed enforcement or better emergency response infrastructure in the identified hotspots.

Count of Accident_id by Accident_severity



c) Correlation Between Traffic Volume and Air Quality Index (AQI)

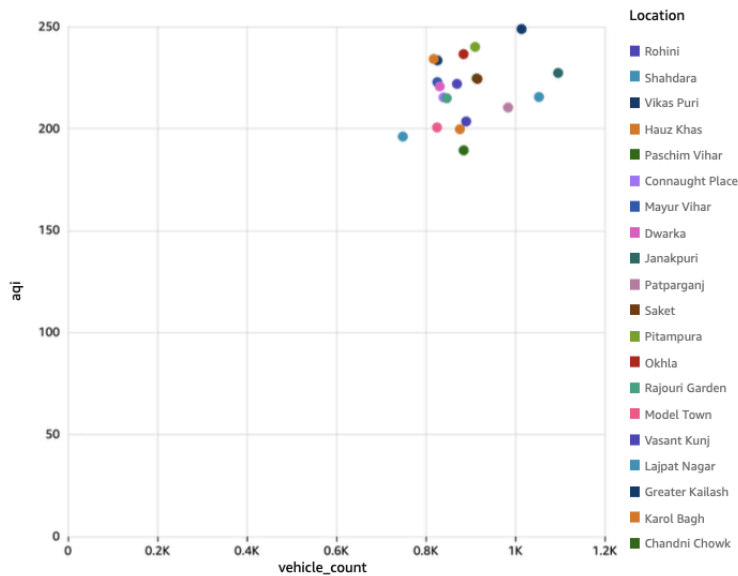
- **Variable Mapping:** Each data point represents a specific Delhi location, plotted by its average daily Vehicle Count (X-axis) against its average AQI (Y-axis).
- **Data Clustering:** Most locations are clustered in the high-traffic/high-pollution quadrant, specifically between 800 to 1,100 vehicles and 180 to 250 AQI.

Insights

- **Positive Correlation:** The upward-sloping trend confirms that as vehicle volume increases, air quality significantly deteriorates, validating traffic as a primary source of urban pollution.
- **Severe Air Quality Threshold:** Multiple locations (like Vikas Puri and Pitampura) have crossed the 200 AQI mark, classifying them as "Poor" or "Very Poor" air quality zones that coincide with peak vehicle densities.

Average of Vehicle_count and Average of Aqi by Location

SHOWING UP TO 2500 DATA POINTS



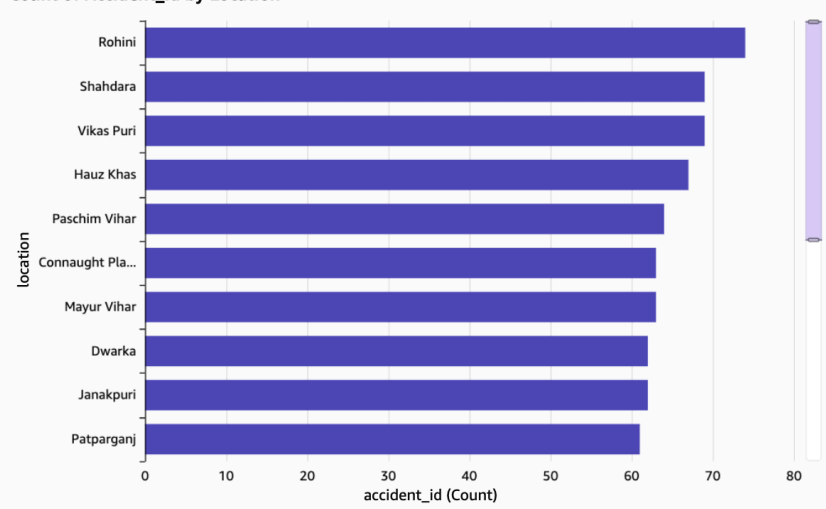
d) Ranking of Traffic Accidents by Location

- **Metric Representation:** The chart displays the total number of recorded accidents (**accident_id** Count) across various major localities in Delhi.
- **Geographic Comparison:** Locations are ranked in descending order, allowing for an immediate visual comparison of safety performance between neighborhoods.

Insights

- **High-Risk Hotspots:** Rohini emerges as the most accident-prone area in the dataset with over 70 incidents, followed closely by Shahdara and Vikas Puri, indicating these zones require priority infrastructure safety audits.
- **Volume Distribution:** The consistent accident counts across diverse locations like Mayur Vihar and Dwarka (mostly ranging between 60–65) suggests that traffic safety risks are a systemic city-wide issue rather than being confined to a single district.

Count of Accident_id by Location

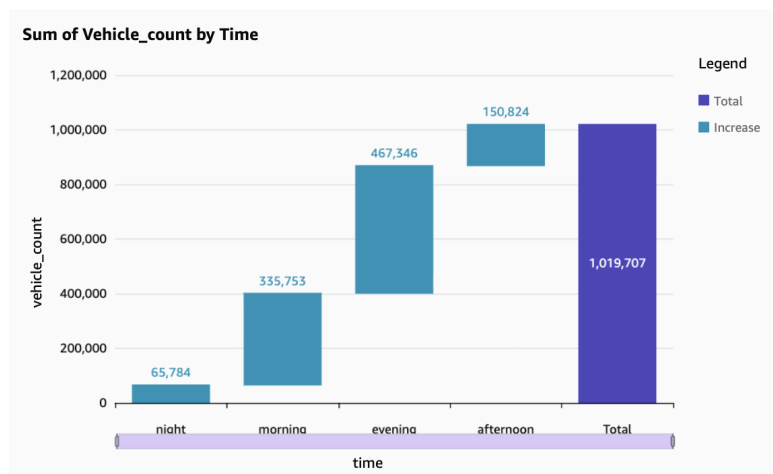


e) Cumulative Vehicle Volume Analysis by Time of Day

- **Incremental Build-up:** The chart uses a "waterfall" effect to show how each time period (Night to Afternoon) contributes incrementally to the city's total traffic volume.
- **Final Aggregation:** The final purple bar represents the Grand Total of over 1 million vehicles recorded across all time segments in the dataset.

Insights

- **Primary Volume Driver:** The Evening shift is the single largest contributor to city traffic, adding 467,346 vehicles—nearly triple the volume added during the Afternoon.
- **Systemic Traffic Loading:** There is a massive surge between Night and Morning (over 500% increase), indicating that the city's infrastructure transitions from near-empty to heavy load almost instantaneously during the start of the workday.



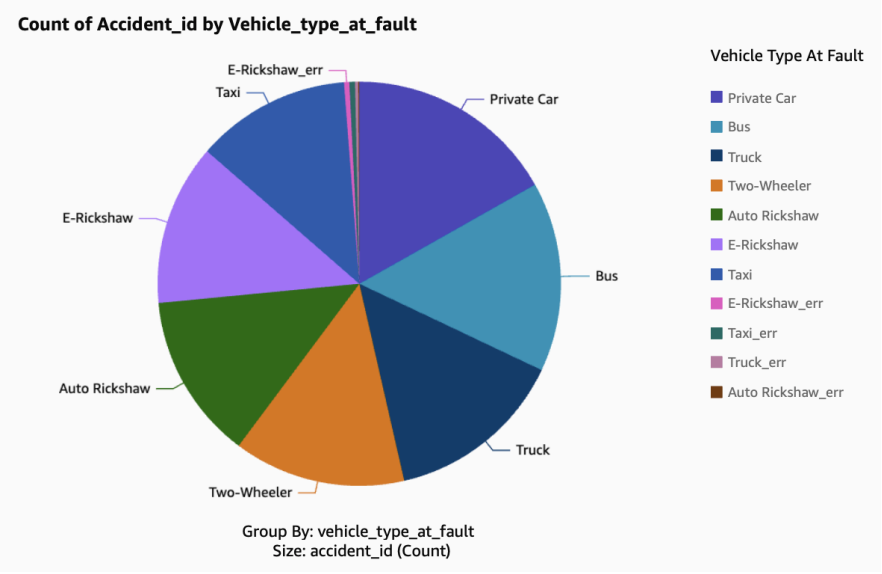
f) Analysis of Accidents by Vehicle Type at Fault

Interpretation

- **Distribution Mapping:** The chart illustrates the proportion of total accidents attributed to various vehicle categories, ranging from private cars and buses to e-rickshaws and two-wheelers.
- **Volume Segmenting:** Each slice represents the count of `accident_id` linked to a specific vehicle type, identifying which modes of transport contribute most to road incidents.

Insights

- **Primary Offenders:** Private Cars, Buses, and Trucks constitute the largest segments of the chart, indicating that heavy and high-speed vehicles are the dominant factors in city-wide accidents.
- **Vulnerable Paratransit:** A significant combined share is held by Auto-Rickshaws and E-Rickshaws, highlighting a need for stricter safety regulations and dedicated lanes for last-mile connectivity vehicles.

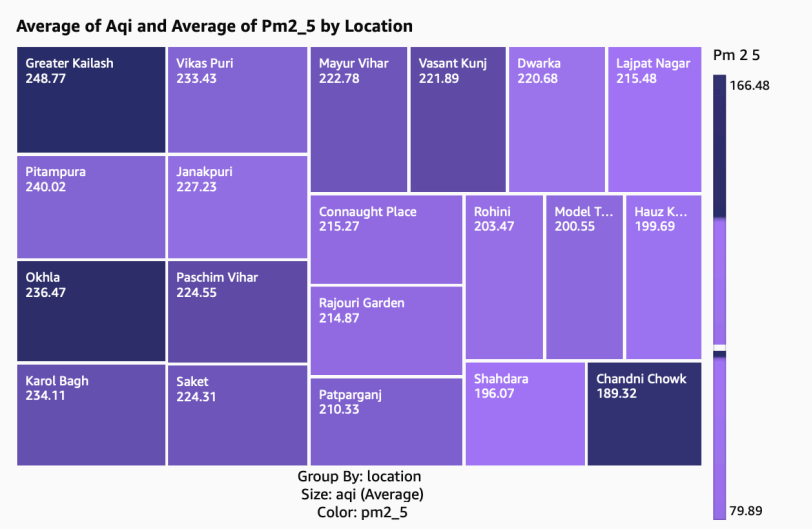


g) Comparative Analysis of AQI and PM2_5 Levels by Location

- **Area Representation (Size):** The size of each rectangular block represents the Average AQI for that location; larger blocks indicate higher overall pollution levels (e.g., Greater Kailash).
- **Color Gradient (Intensity):** The color represents the concentration of $PM_{2.5}$ particles, with darker shades indicating higher density (ranging from 79.89 to 166.48).

Insights

- **Primary Pollution Hotspots:** Greater Kailash and Okhla stand out as the most critical zones, as they combine large physical areas (high AQI) with the darkest color saturation (peak $PM_{2.5}$ levels).
- **Fine Particle Variance:** While several locations have similar AQI scores (like Lajpat Nagar and Connaught Place), their varying color shades reveal that the *type* of pollutants differs, likely due to local factors like industrial exhaust versus commercial traffic.

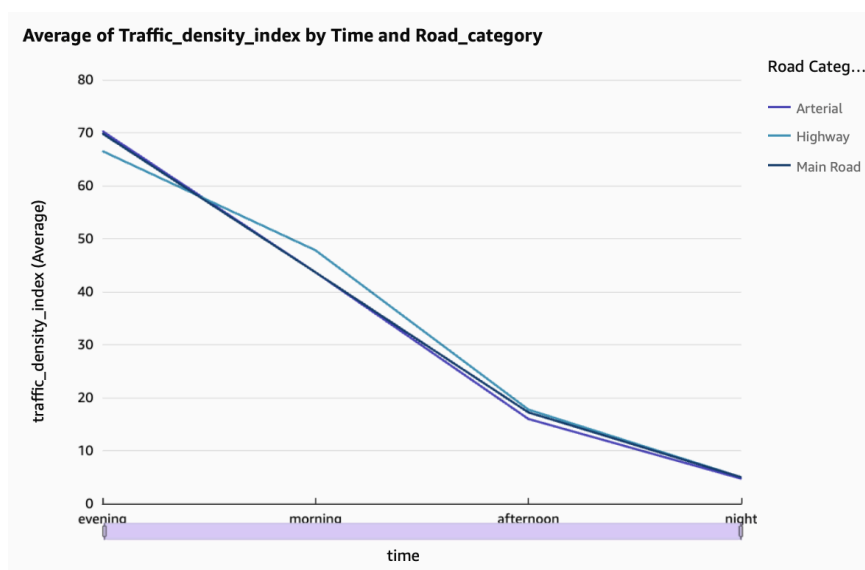


h) Temporal Variations in Traffic Density by Road Category

- **Variable Correlation:** The chart tracks the Average Traffic Density Index (Y-axis) across different times of the day (X-axis), categorized by road types: Arterial, Highway, and Main Road.
- **Trend Direction:** All road categories show a consistent downward trajectory as the day progresses from evening and morning peaks toward afternoon and night lows.

Insights

- **Peak Loading Symmetry:** Both Arterial and Main Roads experience their highest congestion levels during the evening (index ~70), while Highways maintain a slightly higher relative density through the morning period, likely due to inter-city freight movement.
- **Nocturnal Fluidity:** By night, traffic density across all categories converges to a near-zero index, indicating that the city's road infrastructure is significantly underutilized during late-night hours, presenting an opportunity for scheduled maintenance or heavy freight transit.



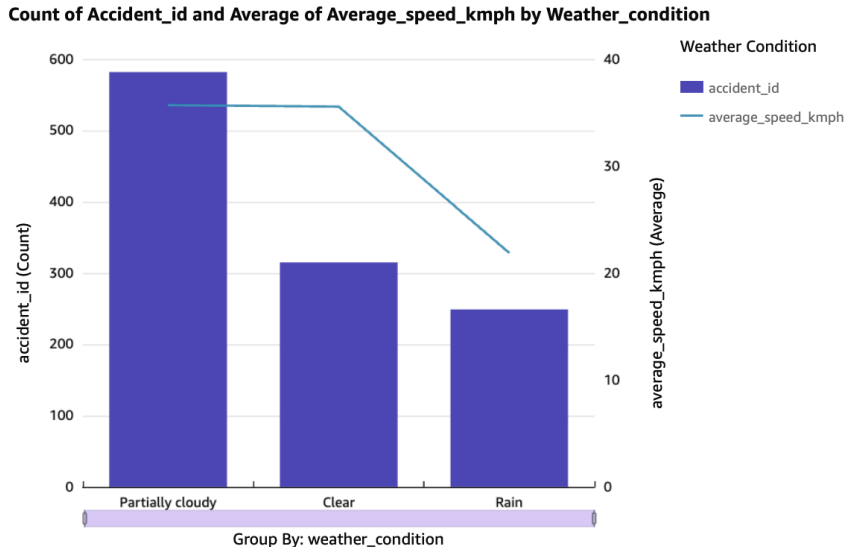
i) Impact of Weather on Accident Frequency and Vehicle Speed

- **Dual-Axis Representation:** The purple bars measure the total Accident Count (left Y-axis), while the blue line tracks the Average Speed in kmph (right Y-axis).
- **Categorical Comparison:** The chart compares these two metrics across three distinct weather states: Partially cloudy, Clear, and Rain.

Insights

- **Weather-Risk Paradox:** Surprisingly, the highest number of accidents (nearly 600) occurs during Partially cloudy conditions, which is significantly higher than during Rain, likely due to higher traffic volumes or driver overconfidence in non-extreme weather.

- **Speed-Safety Correlation:** There is a sharp decline in average speed during Rain (dropping to approx. 22 kmph). This suggests that while bad weather slows down the city, it may actually act as a natural speed regulator, resulting in fewer overall accidents compared to "better" weather days.

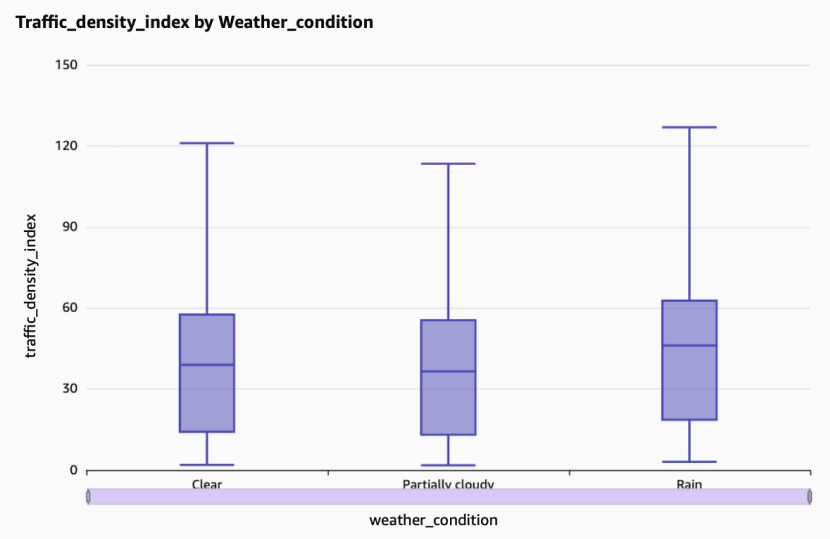


j) Analysis of Traffic Density Distribution by Weather Condition

- **Distribution Range:** The "boxes" represent the middle 50% of the traffic density data, while the "whiskers" show the full range from minimum to maximum recorded values for each weather type.
- **Median Comparison:** The horizontal line inside each box indicates the median traffic density, allowing for a quick comparison of "typical" traffic conditions during Clear, Partially cloudy, and Rain events.

Insights

- **Impact of Rain on Volatility:** Traffic during Rain exhibits a higher median and a much taller box compared to other conditions, indicating that rainfall not only increases overall congestion but also makes traffic levels highly unpredictable.
- **Extreme Outliers:** All three categories show significantly high upper whiskers (reaching indices above 110), proving that extreme gridlock events can occur regardless of weather, though the highest peak is observed during rainy conditions.



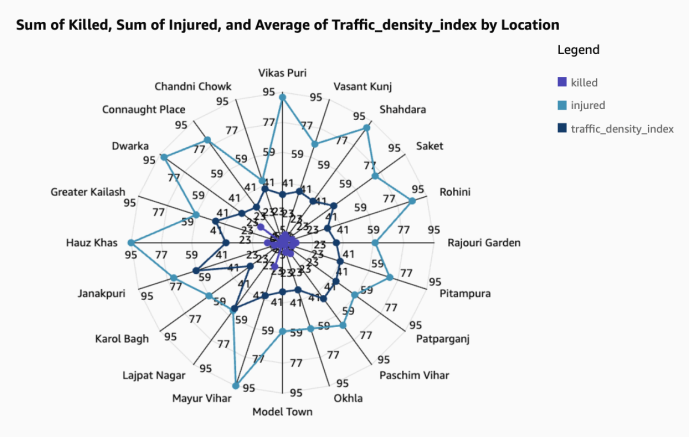
k) Comparative Safety and Traffic Profile by Location

Interpretation

- **Multivariate Analysis:** The chart plots three distinct metrics—Killed, Injured, and Traffic Density Index—across 20 different Delhi locations on a single circular grid.
- **Radial Scaling:** Each "arm" represents a location, with data points closer to the outer edge (95+) signifying higher values, while points near the center indicate lower figures.

Insights

- **Casualty vs. Density Disconnect:** Locations like Vikas Puri and Mayur Vihar show significant spikes in the "Injured" metric (reaching near the 95-mark), despite having moderate traffic density, suggesting these areas are prone to high-impact collisions.
- **Core Safety Stability:** The "Killed" metric (innermost purple cluster) remains relatively centralized and low across all locations compared to injuries, indicating that while road incidents are frequent and result in injuries, fatal outcomes are comparatively less frequent across the surveyed grid.

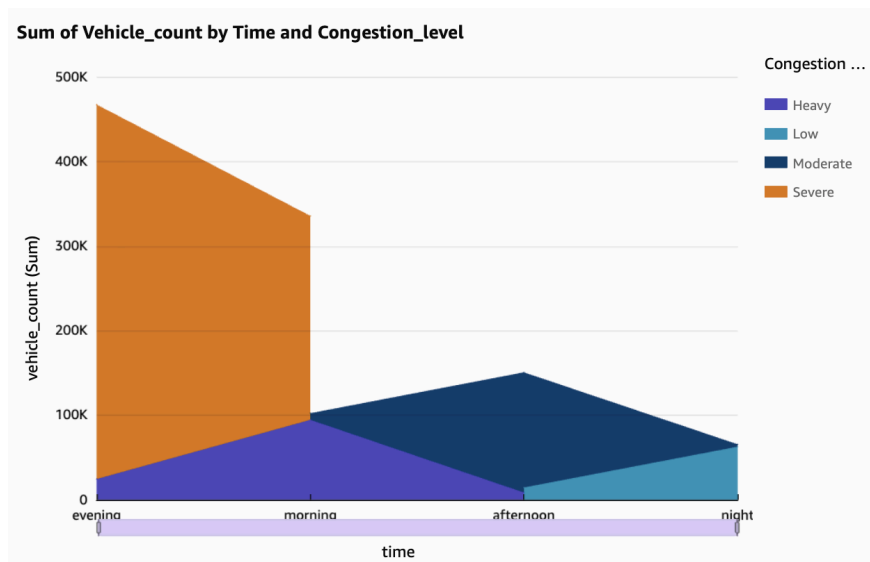


l) Temporal Distribution of Vehicle Volume by Congestion Level

- **Volume Composition:** The chart uses colored areas to show how the total vehicle count is divided into four congestion categories: Severe, Heavy, Moderate, and Low.
- **Temporal Flow:** It tracks the shift in these volumes across the day, from the high-traffic evening and morning peaks to the lower-volume afternoon and night periods.

Insights

- **Peak Crisis Intensity:** The Evening period is dominated by "Severe" congestion (orange), accounting for the vast majority of the city's 450k+ vehicle volume at that time, indicating a near-total breakdown of traffic flow during the commute home.
- **Mid-Day Shift:** By the Afternoon, the congestion profile shifts significantly toward "Moderate" (dark blue) and "Low" (light blue) levels, showing that while vehicle counts remain over 100k, the city's infrastructure manages the load much more effectively than during peak hours.



8. CONCLUSION :

The Delhi Smart City Transportation Analysis project successfully demonstrates the power of a cloud-native, serverless architecture in solving complex urban challenges. By leveraging the AWS Ecosystem, we transformed millions of rows of raw, noisy IoT telemetry into a high-performance analytical engine and a real-time executive dashboard.

Project Achievements :

1. **Scalability & Resilience:** Through the use of Amazon S3 and AWS Lambda, we built a pipeline that scales automatically with the city's data volume, ensuring no "lag" between data ingestion and insight generation.
2. **Data Integrity:** The implementation of the IQR (Interquartile Range) method within the processing layer ensured that the final analysis was based on mathematically

sound, "clean" data, free from sensor malfunctions.

3. **Integrated Intelligence:** By utilizing Amazon Athena to create a Master Query Table, we broke down data silos. We proved that traffic congestion is not an isolated event but is deeply correlated with environmental factors like air quality and weather conditions.
4. **Actionable Visualization:** The Amazon QuickSight dashboards provide city authorities with a "Single Pane of Glass" to monitor Delhi's mobility health, enabling faster response times for emergency services and better long-term urban planning.

9. FUTURE SCOPE :

While the current pipeline provides historical and near-real-time insights, the next evolution of this project involves:

1. **Predictive Analytics:** Integrating AWS SageMaker to predict traffic jams 30–60 minutes before they occur.
2. **Edge Computing:** Using AWS IoT Greengrass to process data directly at the traffic lights to reduce latency even further.
3. **Public Integration:** Developing a mobile API to share these real-time "Traffic-Weather" risk scores directly with the citizens of Delhi.